KI PRAXIS

Die Handbuchreihe für produktives Arbeiten mit künstlicher Intelligenz

BAND 4

Lokale KI und eigene KI-Systeme

Sprachmodelle lokal betreiben und eigene KI-Assistenten entwickeln

Eugen Becker

1. Auflage, November 2026

Impressum und Nutzungshinweise

Reihe KI PRAXIS, Band 4: Lokale KI und eigene KI-Systeme. Sprachmodelle lokal betreiben und eigene KI-Assistenten entwickeln.

Herausgeber und Autor: Eugen Becker.

1. Auflage, November 2026. Alle Rechte vorbehalten. Das Werk einschließlich aller Teile ist urheberrechtlich geschützt. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes bedarf der Zustimmung des Herausgebers.

Hinweis zu den Codebeispielen: Die Programme dieses Bandes laufen mit Python 3 und verwenden nur die Standardbibliothek. Sie wurden in einer Testumgebung ausgeführt, in der kein echtes Sprachmodell geladen werden konnte. Die Ausgaben im Buch stammen deshalb von einem Testserver, der die Schnittstelle eines lokalen Modellservers nachahmt, und nicht von einem Sprachmodell. Der Client-Teil ist gegen diese Schnittstellenbeschreibung geprüft. Die Antworten inhaltlich zu beurteilen, ist mit echten Modellen Aufgabe des Lesers.

Angaben zu Modellservern, Befehlen, Oberflächen, Modellnamen und Hardwareanforderungen sind versionsabhängig und ändern sich häufig. Prüfe vor dem Einsatz die aktuelle Dokumentation des jeweiligen Projekts.

Gesetze und Richtlinien werden in dem Stand zitiert, der zum Zeitpunkt der Drucklegung galt. Maßgeblich ist immer die aktuell gültige Fassung.

Vorwort

Die bisherigen Bände sind davon ausgegangen, dass Sprachmodelle über Dienste fremder Anbieter genutzt werden. Dieser Band dreht die Perspektive um. Er zeigt, wie du ein Modell auf eigener Hardware betreibst, wie du es in eine Oberfläche oder ein Programm einbindest und wie du daraus einen Assistenten baust, der mit deinen eigenen Dokumenten arbeitet.

Lokale Systeme haben einen Reiz, der sich nüchtern begründen lässt. Vertrauliche Unterlagen verlassen das eigene Netz nicht. Die Kosten sind planbar, und die Verfügbarkeit hängt nicht von einem Anbieter ab. Zugleich verlangen sie Hardware, Pflege und Sicherheitsarbeit, die ein Cloud-Dienst dir abnimmt. Dieser Band nennt beide Seiten und hilft dir, ehrlich abzuwägen.

Vorausgesetzt werden die Kenntnisse aus den Bänden 2 und 3, insbesondere die Konzepte von RAG und Schnittstellen. Die Codebeispiele sind klein gehalten, damit du jede Zeile verstehst. Für die Kapitel 7 und 8 brauchst du einen Rechner oder Server, auf dem du Dienste einrichten darfst.

Einleitung und Zielgruppe

Dieser Band richtet sich an Fachkräfte in Verwaltung, Mittelstand und Technik, an Lehrende und an alle, die ihre Daten nicht dauerhaft in fremde Dienste geben wollen. Er setzt Grundkenntnisse im Umgang mit einem Computer voraus. Für die Kapitel 3 bis 6 hilft Erfahrung mit einem Terminal und mit Python.

Der Schwierigkeitsgrad steigt an. Die Kapitel 1 und 2 sind für alle gut zugänglich und helfen bei der Entscheidung, ob ein lokales System sinnvoll ist. Die Kapitel 5 und 6 verbinden mehrere Konzepte, und das Kapitel 7 setzt Administrationserfahrung voraus.

Die Hinweisboxen aus den früheren Bänden kehren zurück. Achtung-Kästen sind in diesem Band besonders wichtig, weil Fehler bei Netzwerk, Zugriffsrechten und Backups lange unbemerkt bleiben können.

Lernziele

- Du kannst den Unterschied zwischen Cloud-KI und lokaler KI begründet darstellen und die Vor- und Nachteile abwägen.
- Du schätzt den Speicherbedarf eines Modells anhand von Parametern und Quantisierung und planst die passende Hardware.
- Du installierst einen lokalen Modellserver, verwaltest Modelle und ordnest Modellparameter sinnvoll zu.
- Du sprichst einen lokalen Modellserver über seine Schnittstelle an und verwaltest mehrere Modelle.
- Du baust einen Assistenten mit Systemprompt, Gesprächsverlauf und Speicherung.

- Du erklärst die Schritte einer lokalen RAG-Pipeline und erkennst, wo einfache Verfahren an Grenzen stoßen.
- Du beschreibst die Grundlagen von Betrieb, Netzwerkzugriff, Benutzerverwaltung und Datensicherung für einen lokalen Server.

Inhaltsverzeichnis

Kapitel 1: Grundlagen lokaler KI **9**

1.1 Was sind lokale Sprachmodelle? 9

1.2 Open-Source- und Open-Weight-Modelle 9

1.3 Cloud-KI und lokale KI im Vergleich 10

1.4 Vorteile und Nachteile 11

Kapitel 2: Hardware und Systemanforderungen **13**

2.1 CPU und GPU 13

2.2 RAM und VRAM 13

2.3 Quantisierung 14

2.4 Modellgrößen 14

2.5 Hardwareplanung 14

Kapitel 3: Lokale KI installieren **17**

3.1 Ein lokaler Modellserver 17

3.2 Modellverwaltung 17

3.3 Modelle herunterladen 18

3.4 Modellparameter 18

3.5 Ressourcenüberwachung 18

Kapitel 4: Benutzeroberflächen und APIs **20**

4.1 Oberflächen für lokale Modelle 20

4.2 Lokale Schnittstellen 20

4.3 REST-API im eigenen Programm 21
4.4 Modellkonfiguration 23
4.5 Mehrere Modelle verwalten 23

Kapitel 5: Eigene KI-Assistenten 26

5.1 Architektur eines Assistenten 26
5.2 Systemprompts 26
5.3 Gedächtnis und Kontext 26
5.4 Gesprächsverwaltung 27
5.5 Werkzeuganbindung 27
5.6 Ein Assistent mit Gedächtnis 27

Kapitel 6: Dokumente mit lokaler KI verarbeiten 30

6.1 Lokale Dokumentenanalyse 30
6.2 RAG-Systeme 30
6.3 Embedding-Modelle 30
6.4 Vektordatenbanken 31
6.5 Lokale Wissensdatenbanken 31
6.6 Das RAG-Beispiel 31
6.7 Grenzen einfacher Verfahren 34

Kapitel 7: Betrieb und Administration 36

7.1 Linux-Server 36
7.2 Docker 36
7.3 Netzwerkzugriff 37
7.4 Benutzerverwaltung 37

7.5 Backups 38
7.6 Monitoring 38
7.7 Datenschutz und Dokumentation 38

Kapitel 8: Praxisprojekte **41**

8.1 Hinweise zu den Projekten 41
8.2 Projekt 1: Eigener lokaler Chatbot 41
8.3 Projekt 2: Lokaler Dokumentenassistent 42
8.4 Projekt 3: Persönlicher KI-Assistent mit Gedächtnis 43

Musterlösungen **45**

Anhang: Betriebscheckliste für lokale KI-Systeme **48**

Glossar **49**

Quellenverzeichnis **51**

Stichwortverzeichnis **52**

Kapitel 1: Grundlagen lokaler KI

1.1 Was sind lokale Sprachmodelle?

Ein lokales Sprachmodell ist ein Modell, das auf einem Rechner läuft, den du kontrollierst. Die Datei mit den Gewichtungen liegt auf deiner Festplatte, und die Berechnung findet auf deiner Hardware statt. Für eine Anfrage verlässt nichts das Gerät, sofern du keine Verbindung nach außen einrichtest. Das unterscheidet lokale Modelle grundlegend von Diensten, bei denen deine Eingaben an einen Anbieter übertragen werden.

Technisch ähnelt ein lokales Modell einem Cloud-Modell. Es verarbeitet Text in Zahlen, rechnet mit seinen Gewichtungen und erzeugt Wort für Wort eine Fortsetzung. Die Unterschiede liegen in Größe, Geschwindigkeit und Qualität, und die hängen stark von der Hardware ab. Ein großes Cloud-Modell in einem Rechenzentrum ist meist leistungsfähiger als das größte Modell, das auf einem Arbeitsplatzrechner läuft.

1.2 Open-Source- und Open-Weight-Modelle

Für lokalen Betrieb kommen Modelle infrage, deren Gewichtungen öffentlich verfügbar sind. Man spricht von Open-Weight-Modellen, wenn die Gewichtungen zum Herunterladen bereitstehen, aber die Lizenz Einschränkungen enthält. Von Open-Source-Modellen spricht man im engeren Sinn, wenn Lizenz und Bedingungen einer anerkannten Open-Source-Lizenz entsprechen. Die Bezeichnungen werden in der Praxis uneinheitlich verwendet, deshalb solltest du die Lizenz jedes Modells selbst lesen.

Achtung: Lizenz und Nutzung

Eine Lizenz kann kommerzielle Nutzung, Weitergabe oder den Einsatz für bestimmte Zwecke einschränken. Lies die Lizenz eines Modells, bevor du es in einem Arbeitsumfeld einsetzt. Die Lizenz gilt für das Modell, nicht für die Software, mit der du es betreibst.

1.3 Cloud-KI und lokale KI im Vergleich

Die Wahl zwischen Cloud und lokal ist eine Abwägung und keine Glaubensfrage. Cloud-Dienste bieten meist die leistungsfähigsten Modelle, sofort verfügbare Aktualisierungen und keine eigene Hardware. Dafür übertragst du Daten an einen Dienst, bist von dessen Preisen und Bedingungen abhängig und kannst nicht überprüfen, wie er intern arbeitet. Lokale Systeme bieten Kontrolle über Daten und Kosten, verlangen aber Hardware, Wissen und Zeit für den Betrieb.

Gesichtspunkt	Cloud-Dienst	Lokales System
Datenhaltung	Beim Anbieter	Auf eigener Hardware
Leistung	Oft die größten Modelle	Durch Hardware begrenzt
Kosten	Laufend, nach Nutzung	Anschaffung und Betrieb
Betrieb	Übernimmt der Anbieter	Übernimmst du
Verfügbarkeit	Abhängig vom Anbieter	Abhängig von deinem Betrieb

1.4 Vorteile und Nachteile

Der wichtigste Vorteil ist die Datenhoheit. Wer Verträge, Personalakten oder Entwicklungsdaten auswertet, gewinnt viel Sicherheit, wenn die Verarbeitung im eigenen Netz bleibt. Weitere Vorteile sind die Unabhängigkeit von Preisänderungen und Dienstausfällen sowie die Möglichkeit, ein Modell für eigene Zwecke anzupassen. Der wichtigste Nachteil ist der Aufwand. Hardware kostet Geld, Aktualisierungen müssen eingespielt werden, und die Qualität kleinerer Modelle reicht für anspruchsvolle Aufgaben oft nicht aus.

Für einen ehrlichen Vergleich lohnt es sich, den Aufwand in Stunden zu schätzen. Die Einrichtung eines lokalen Systems dauert einen Nachmittag, die Pflege über das Jahr hinweg eher mehrere Tage. Diese Rechnung ergibt sich aus deiner Lage und ist nicht allgemein gültig.

Schritt für Schritt: Entscheidung für oder gegen ein lokales System

1. Beschreibe die Daten, die verarbeitet werden sollen, und ihre Schutzbedürftigkeit.
2. Notiere die Aufgaben und die Qualität, die du dafür brauchst.
3. Schätze den Aufwand für Hardware, Einrichtung und laufenden Betrieb in Stunden und Euro.
4. Vergleiche die Ergebnisse mit einem Cloud-Angebot, das die Datenschutzanforderungen erfüllt.
5. Entscheide, und notiere die Begründung, damit du sie später überprüfen kannst.

Erwartetes Ergebnis: Eine begründete Entscheidung mit einer Schätzung des Aufwands, die du in einem Jahr noch nachvollziehen kannst.

Typische Fehler

Der häufigste Fehler ist, ein lokales System allein aus Datenschutzgründen zu wählen, ohne den Betrieb zu planen. Ein unbetreutes Modell auf einem Rechner ohne Updates und Sicherung ist ein größeres Risiko als ein gut betreuter Cloud-Dienst. Ein zweiter Fehler ist die Erwartung, ein kleines lokales Modell könne die Leistung eines großen Cloud-Modells erreichen.

Kurz gesagt

Lokale Modelle geben dir Kontrolle über Daten und Kosten und verlangen dafür Hardware und Betrieb. Die Entscheidung sollte auf einer ehrlichen Aufwandsschätzung beruhen.

Übungen

Übung 1.1: Notiere für einen Anwendungsfall aus deinem Alltag, welche Daten anfallen und welche Schutzbedürftigkeit sie haben.

Übung 1.2: Lies die Lizenz eines Modells deiner Wahl und notiere drei Bedingungen, die für deine Nutzung wichtig sind.

Kapitel 2: Hardware und Systemanforderungen

2.1 CPU und GPU

Ein Sprachmodell rechnet bei jeder erzeugten Silbe mit sehr vielen Zahlen. Eine Grafikkarte, GPU, ist für solche parallelen Rechnungen gebaut und arbeitet dabei meist um ein Vielfaches schneller als ein Prozessor, CPU. Ein Modell kann aber auch vollständig auf der CPU laufen, nur langsamer. Für einfache Aufgaben und kurze Antworten reicht das oft. Für lange Dokumente und flüssige Dialoge ist eine GPU mit ausreichend Speicher der übliche Weg.

Die Geschwindigkeit wird bei der Texterzeugung weniger durch die Rechenleistung begrenzt als durch die Geschwindigkeit, mit der die Gewichtungen aus dem Speicher gelesen werden. Man nennt das eine Speicherbandbreite-Grenze. Deshalb hilft eine schnelle Grafikkarte mit viel Speicher oft mehr als eine besonders schnelle CPU mit wenig Speicher.

2.2 RAM und VRAM

Das Modell muss vollständig in den Speicher passen, damit es zügig läuft. Auf einer Grafikkarte ist das der VRAM, auf der CPU der Arbeitsspeicher, RAM. Wenn das Modell nicht passt, lagert das System Teile auf die Festplatte aus, und die Geschwindigkeit fällt stark ab. Zusätzlich braucht jede laufende Sitzung Speicher für den Gesprächsverlauf, der mit der Länge des Kontextfensters wächst.

2.3 Quantisierung

Quantisierung bedeutet, die Gewichtungen mit weniger Bits zu speichern. Ein Modell, dessen Gewichtungen ursprünglich mit 16 Bit vorliegen, lässt sich oft auf 8 oder 4 Bit verkleinern. Der Speicherbedarf sinkt dadurch deutlich, die Qualität sinkt in der Regel nur leicht, bei sehr starker Verkleinerung aber spürbar. Das Verfahren ist ein Kompromiss, und seine Wirkung hängt vom Modell und von der Aufgabe ab.

2.4 Modellgrößen

Die Größe eines Modells wird in Parametern angegeben, meist in Milliarden. Ein Modell mit 7 Milliarden Parametern ist klein, eines mit 70 Milliarden ist groß. Mehr Parameter bedeuten in der Regel bessere Fähigkeiten, aber auch mehr Speicherbedarf und langsamere Antworten. Für viele Alltagsaufgaben sind mittelgroße, gut eingestellte Modelle ausreichend.

2.5 Hardwareplanung

Eine Faustregel hilft beim Überschlag. Der Speicherbedarf eines Modells ergibt sich grob aus der Zahl der Parameter in Milliarden, mal der Zahl der Bits pro Gewicht, geteilt durch acht, mal etwa 1,2 als Zuschlag für Laufzeit und Verwaltung. Ein Modell mit 7 Milliarden Parametern und 4 Bit braucht demnach etwa 3,5 Gigabyte für die Gewichtungen, mit Zuschlag rund 4,2 Gigabyte. Ein Modell mit 13 Milliarden Parametern landet bei etwa 7,8 Gigabyte, eines mit 70 Milliarden bei etwa 42 Gigabyte.

Die Faustregel ist ein Überschlag und ersetzt keine Messung. Architektur, Kontextlänge und die verwendete Software beeinflussen den tatsächlichen Bedarf. Prüfe die Angaben zum Speicherbedarf in der Dokumentation des konkreten Modells und miss den Verbrauch

nach dem ersten Start.

Schritt für Schritt: Hardware für einen Anwendungsfall planen

1. Lege fest, welche Aufgaben das System erfüllen soll und wie lang die Dokumente sind.
2. Wähle eine Modellgröße, die für diese Aufgaben ausreichend erscheint, und eine Quantisierung.
3. Überschlage den Speicherbedarf mit der Faustregel und plane einen Puffer von etwa einem Viertel ein.
4. Prüfe, ob der Rechner oder Server diesen Speicher bereitstellt, und ob der Prozessor die Last bei mehreren Nutzern trägt.
5. Miss nach dem ersten Start den tatsächlichen Verbrauch und passe die Planung an.

Erwartetes Ergebnis: Eine Hardwareplanung mit Überschlag, Puffer und gemessenem Verbrauch, die begründet, warum die Auswahl passt.

Typische Fehler

Der häufigste Fehler ist, nur den Speicher für die Gewichtungen zu planen und den Bedarf für Kontext und mehrere Nutzer zu vergessen. Ein zweiter ist, ein Modell zu wählen, das knapp in den Speicher passt. Dann läuft das System zwar, aber nicht flüssig, und niemand nutzt es gern.

Kurz gesagt

Der Speicher bestimmt, welche Modelle sinnvoll laufen. Die Faustregel mit Parametern, Bits und Zuschlag gibt einen ersten Überschlag, die Messung im Betrieb gibt die Wahrheit.

Übungen

Übung 2.1: Überschlage den Speicherbedarf eines Modells mit 13 Milliarden Parametern bei 8 Bit Quantisierung.

Übung 2.2: Vergleiche zwei Rechner, die dir zur Verfügung stehen, und entscheide, welcher für ein Modell mit 7 Milliarden Parametern geeignet wäre.

Kapitel 3: Lokale KI installieren

3.1 Ein lokaler Modellserver

Ein lokaler Modellserver ist ein Programm, das Modelle lädt, sie für Anfragen bereithält und über eine Schnittstelle erreichbar macht. Oberflächen, Skripte und eigene Programme sprechen dann mit dem Server, nicht direkt mit dem Modell. Dieser Aufbau trennt die Verwaltung der Modelle von ihrer Nutzung und erlaubt, mehrere Anwendungen mit demselben Server zu betreiben.

Ollama ist ein weit verbreiteter Modellserver, der sich auf vielen Betriebssystemen installieren lässt. Die Installation folgt der offiziellen Anleitung des Projekts, die sich ändert. Dieses Buch nennt deshalb keine Installationsschritte im Detail, sondern beschreibt, was du danach prüfen solltest.

3.2 Modellverwaltung

Modelle werden über den Server heruntergeladen, aufgelistet und entfernt. Die Befehle dafür heißen in der aktuellen Version des Projekts sinngemäß herunterladen, auflisten und entfernen. Die genauen Namen der Befehle und ihre Optionen stehen in der Dokumentation und sollten dort nachgeschlagen werden. Ein Modell belegt mehrere Gigabyte Festplatte, deshalb lohnt es sich, nicht genutzte Modelle regelmäßig zu entfernen.

3.3 Modelle herunterladen

Beim Herunterladen wählst du ein Modell mit Namen und Variante. Der Name enthält oft die Größe und die Quantisierung. Achte auf die Lizenz und die Herkunft, bevor du ein Modell einsetzt, und lade Modelle nur aus Quellen, denen du vertraust. Modellkarten, also Beschreibungen der Modelle, geben Auskunft über Trainingsdaten, Einschränkungen und vorgesehene Nutzung.

3.4 Modellparameter

Jede Anfrage lässt sich über Parameter steuern. Die Temperatur bestimmt, wie stark die Antwort vom wahrscheinlichsten Wort abweicht. Niedrige Werte liefern gleichmäßigere, aber eintönigere Antworten, höhere Werte mehr Abwechslung und mehr Fehler. Für Aufgaben, die Genauigkeit verlangen, sind niedrige Werte sinnvoll. Die Länge des Kontextfensters legt fest, wie viel Text das Modell gleichzeitig berücksichtigen kann. Sie beeinflusst den Speicherbedarf direkt.

Die Namen und Zulässigkeiten der Parameter ändern sich zwischen Versionen. Prüfe sie in der Dokumentation, bevor du Werte festlegst.

3.5 Ressourcenüberwachung

Während das Modell läuft, solltest du Speicher, Prozessorlast und Temperatur im Blick behalten. Unter Linux zeigt der Befehl für den freien Speicher die Auslastung des Arbeitsspeichers. Für Grafikkarten der Firma NVIDIA gibt das Werkzeug zur Überwachung der GPU Auskunft über den VRAM und die Auslastung. Andere Hersteller haben eigene Werkzeuge. Notiere den Verbrauch bei typischen Aufgaben, denn daraus ergibt sich die Planung für den Betrieb.

Achtung: Serverdienst nur lokal erreichbar machen

Viele Modellserver lauschen standardmäßig nur auf dem eigenen Rechner. Ändere diese Einstellung nur, wenn du weißt, wer das Netz erreichen kann, und sichere den Zugang ab. Ein offener Modellserver im Netz kann von jedem Gerät genutzt werden, und das kann Rechenzeit und Daten kosten.

Typische Fehler

Der häufigste Fehler nach der Installation ist, ein Modell zu laden und den Speicherverbrauch nie zu messen. Ein zweiter ist, Parameter aus Beispielen zu übernehmen, ohne sie für die eigene Aufgabe zu prüfen. Ein dritter ist, den Server im Netz zu öffnen, weil es in einer Anleitung so stand.

Kurz gesagt

Ein Modellserver trennt Verwaltung und Nutzung der Modelle. Lade nur geprüfte Modelle, passe die Parameter an deine Aufgabe an und behalte Ressourcen im Blick.

Übungen

Übung 3.1: Installiere einen Modellserver nach der offiziellen Anleitung und notiere die Schritte, die du dabei tatsächlich ausgeführt hast.

Übung 3.2: Miss den Speicherverbrauch deines Systems im Leerlauf und während einer Anfrage und vergleiche die Werte.

Kapitel 4: Benutzeroberflächen und APIs

4.1 Oberflächen für lokale Modelle

Nicht jede Aufgabe verlangt ein Programm. Für Gespräche und Tests genügt oft eine Weboberfläche, die mit dem Modellserver spricht. Open WebUI ist ein verbreitetes Projekt dieser Art. Es bietet einen Gesprächsbereich, Verwaltung von Modellen und Benutzerkonten. Die Installation erfolgt in der Regel über Docker, das Kapitel 7 erläutert.

Eine Oberfläche ist bequem, verdeckt aber, was technisch geschieht. Wer nur die Oberfläche kennt, weiß nicht, welcher Verlauf an das Modell geschickt wird und wo er gespeichert ist. Deshalb lohnt es sich, auch die Schnittstelle zu verstehen, auf der die Oberfläche aufbaut.

4.2 Lokale Schnittstellen

Ein Modellserver stellt seine Funktionen über HTTP bereit, also über dieselbe Art von Anfragen, die auch Webseiten nutzen. Eine Anfrage geht an eine Adresse wie der Serveradresse mit einem Pfad, und die Antwort kommt als JSON zurück. Die Adresse besteht aus Rechner und Port. Die Standardadresse lautet auf dem eigenen Rechner, mit einem festen Port. Die genauen Pfade und Felder stehen in der Schnittstellendokumentation des Projekts.

4.3 REST-API im eigenen Programm

Das folgende Modul kapselt die Anfragen an den Server. Es enthält drei Funktionen: eine für die Fehlerbehandlung, eine zum Abrufen der Modellliste und eine für den Chat. Der Chat verlangt eine Liste von Nachrichten mit Rolle und Inhalt, so wie in Band 3 beschrieben. Das Modul ist bewusst klein, damit sich die Schnittstelle leicht an neue Versionen anpassen lässt.

Codebeispiel 4.1: Client für die REST-Schnittstelle

```
import json
import urllib.error
import urllib.request

BASIS = "http://127.0.0.1:11434"


class ModellFehler(Exception):
    pass


def anfrage(pfad, daten=None, timeout=120):
    body = None
    methode = "GET"
    if daten is not None:
        body = json.dumps(daten).encode("utf-8")
        methode = "POST"
    req = urllib.request.Request(
        BASIS + pfad, data=body, method=methode,
        headers={"Content-Type": "application/json"})
    try:
        with urllib.request.urlopen(req, timeout=timeout) as antwort:
            return json.loads(antwort.read())
    except urllib.error.HTTPError as fehler:
        meldung = json.loads(fehler.read()).get("error", "")
        raise ModellFehler(f"HTTP {fehler.code}: {meldung}")
    except urllib.error.URLError as fehler:
        raise ModellFehler(f"Server nicht erreichbar: {fehler.reason}")


def modelle():
    return [m["name"] for m in anfrage("/api/tags")["models"]]


def chat(modell, nachrichten, temperatur=0.2):
    antwort = anfrage("/api/chat", {
        "model": modell,
        "messages": nachrichten,
        "stream": False,
        "options": {"temperature": temperatur},
    })
    return antwort["message"]["content"]
```

Ausgabe

```
Modelle: ['test-modell:latest']
Antwort: Platzhalter: 1 Nachrichten, System nein, letzte 5 Zeichen
Fehler: HTTP 404: model 'gibt-es-nicht' not found
```

Die Ausgabe stammt aus dem Testlauf. Der erste Teil zeigt die Modellliste und eine Antwort des Testservers. Der zweite Teil zeigt, wie ein unbekanntes Modell mit dem Statuscode 404 abgelehnt wird. Die Fehlermeldung landet in einer eigenen Ausnahme, die dein Programm gezielt abfangen kann. Ein echter Modellserver liefert Antworten inhaltlich anders, die Struktur des Aufrufs bleibt gleich.

4.4 Modellkonfiguration

Die Konfiguration eines Modells besteht aus den Parametern aus Kapitel 3 und aus dem Systemprompt, den du in Kapitel 5 kennenlernst. Lege Standardwerte an einer Stelle fest, zum Beispiel in einer Konfigurationsdatei, damit alle Anwendungen dieselben Werte nutzen. Unterschiedliche Aufgaben verlangen oft unterschiedliche Temperaturen, und das lässt sich über eigene Konfigurationen abbilden.

4.5 Mehrere Modelle verwalten

Viele Installationen brauchen mehr als ein Modell. Ein kleines Modell beantwortet einfache Fragen schnell, ein größeres übernimmt anspruchsvolle Aufgaben. Mehrere Modelle belegen Festplattenplatz, und gleichzeitig geladene Modelle belegen Speicher. Lege deshalb fest, welche Modelle dauerhaft bereitstehen und welche bei Bedarf geladen werden. Eine kleine Tabelle mit Modell, Aufgabe, Speicherbedarf und Zuständigkeit verhindert Verwirrung.

Schritt für Schritt: Eine Anwendung an den Modellserver anbinden

1. Prüfe, unter welcher Adresse der Server erreichbar ist, und notiere sie in einer Konfiguration.
2. Rufe die Modellliste ab und bestätige, dass das gewünschte Modell vorhanden ist.
3. Sende eine einfache Testanfrage und prüfe die Antwort auf Inhalt und Format.
4. Fange Fehler gezielt ab und entscheide, wann ein Fehler dem Benutzer gezeigt wird.
5. Dokumentiere die Konfiguration, damit andere Anwendungen dieselben Werte übernehmen können.

Erwartetes Ergebnis: Eine Anwendung, die das gewünschte Modell findet, eine Testanfrage beantwortet und bei Fehlern verständlich reagiert.

Typische Fehler

Der häufigste Fehler ist, Adressen und Ports fest in den Code zu schreiben, sodass ein Wechsel des Servers Änderungen in jeder Anwendung verlangt. Ein zweiter ist, Fehlermeldungen des Servers ungeprüft anzuzeigen, obwohl sie technische Details enthalten, die nicht für Benutzer bestimmt sind.

Kurz gesagt

Oberflächen sind bequem, die Schnittstelle ist das eigentliche Werkzeug. Ein sauberer Client kapselt die Anfragen, fängt Fehler gezielt ab und liest die Konfiguration aus einer Stelle.

Übungen

Übung 4.1: Ändere im Client die Zeitgrenze auf fünf Sekunden und notiere, wie sich das Verhalten bei einem nicht erreichbaren Server ändert.

Übung 4.2: Lege eine Tabelle mit drei Modellen an, jeweils mit Aufgabe und Speicherbedarf, und begründe, welches Modell für welche Aufgabe gewählt wird.

Kapitel 5: Eigene KI-Assistenten

5.1 Architektur eines Assistenten

Ein Assistent ist mehr als ein Modell mit einer Eingabe. Er besteht aus dem Modell, einer Anweisung, die sein Verhalten festlegt, einem Gesprächsverlauf, einer Ablage für frühere Gespräche und gegebenenfalls Werkzeugen, die er nutzen darf. Jeder dieser Teile lässt sich einzeln entwerfen, testen und ersetzen.

5.2 Systemprompts

Der Systemprompt ist eine Anweisung, die bei jeder Anfrage mitgesendet wird und die Rolle und Grundregeln des Assistenten festlegt. Er bestimmt Ton, Länge, Sprache und Grenzen. Ein guter Systemprompt ist kurz, konkret und enthält nur Regeln, die für alle Gespräche gelten. Aufgabenspezifische Angaben gehören in die Nachricht des Benutzers.

Der Systemprompt ist keine Sicherheitsgrenze. Ein Modell kann sich durch geschickte Eingaben über Anweisungen hinwegsetzen. Wichtige Grenzen musst du deshalb im Programm absichern, etwa durch erlaubte Werkzeuge und Freigaben, wie in Band 3 beschrieben.

5.3 Gedächtnis und Kontext

Ein Modell erinnert sich nicht selbst an frühere Gespräche. Das Gedächtnis entsteht, indem das Programm den bisherigen Verlauf bei jeder Anfrage erneut mitsendet. Da das Kontextfenster begrenzt ist, muss das Programm entscheiden, welche Nachrichten es behält. Die einfachste Lösung ist, nur die letzten Nachrichten mitzuschicken. Der Systemprompt bleibt dabei immer erhalten.

5.4 Gesprächsverwaltung

Ein Gespräch sollte sich speichern und wieder laden lassen. Das ermöglicht, an einem Thema weiterzuarbeiten, ohne alles erneut einzugeben. Speichere den Verlauf in einer Datei, die du kontrollierst, und überlege, welche Inhalte du dauerhaft ablegen willst. Verläufe enthalten oft personenbezogene Angaben, für die die Datenschutz-Grundverordnung gilt.

5.5 Werkzeuganbindung

Ein Assistent kann auf Werkzeuge zugreifen, etwa einen Kalender, eine Datei oder eine Datenbank. Dafür gilt, was Band 3 für Agenten beschreibt: nur erlaubte Werkzeuge, Freigabe für wirksame Aktionen und ein Protokoll. Der Einstieg sollte mit einem Assistenten ohne Werkzeuge beginnen, um Gesprächsführung und Verlauf sauber zu beherrschen.

5.6 Ein Assistent mit Gedächtnis

Das folgende Modul verwaltet ein Gespräch. Es hält den Verlauf, begrenzt die Zahl der mitgesendeten Nachrichten, schreibt den Systemprompt bei jeder Anfrage dazu und kann den Verlauf in eine Datei speichern und wieder laden. Die Begrenzung ist im Test auf vier Nachrichten gesetzt, damit du ihre Wirkung sehen kannst.

Codebeispiel 5.1: Gesprächsverlauf mit Begrenzung und Speicherung

```
import json

from lokal_client import chat

SYSTEM = {
    "role": "system",
    "content": "Du bist ein sachlicher Assistent. "
               "Antworte knapp auf Deutsch.",
}


class Gespraech:
    def __init__(self, modell, max_nachrichten=6):
        self.modell = modell
        self.max_nachrichten = max_nachrichten
        self.verlauf = []

    def frage(self, text):
        self.verlauf.append({"role": "user", "content": text})
        letzte = self.verlauf[-self.max_nachrichten:]
        antwort = chat(self.modell, [SYSTEM] + letzte)
        self.verlauf.append({"role": "assistant", "content": antwort})
        return antwort

    def speichern(self, pfad):
        with open(pfad, "w", encoding="utf-8") as datei:
            json.dump(self.verlauf, datei,
                      ensure_ascii=False, indent=2)

    def laden(self, pfad):
        with open(pfad, encoding="utf-8") as datei:
            self.verlauf = json.load(datei)
```

Ausgabe

```
Platzhalter: 2 Nachrichten, System ja, letzte 11 Zeichen
Platzhalter: 4 Nachrichten, System ja, letzte 12 Zeichen
Platzhalter: 5 Nachrichten, System ja, letzte 12 Zeichen
Platzhalter: 5 Nachrichten, System ja, letzte 12 Zeichen
Geladene Nachrichten: 8
```

Die Ausgabe zeigt, wie viele Nachrichten pro Anfrage ankommen. Bei der ersten Frage sind es zwei, der Systemprompt und die Frage. Ab der dritten Frage bleibt die Zahl konstant, weil die Begrenzung greift. Das Modell sieht dann nur die letzten Nachrichten. Der letzte Teil zeigt, dass der gespeicherte Verlauf acht Nachrichten umfasst, also vier

Frage-Antwort-Paare, und vollständig geladen werden kann.

> **Achtung: Gesprächsverläufe speichern**
> Gespräche enthalten häufig Namen, Adressen oder Gesundheitsangaben, die der Nutzer nebenbei erwähnt. Lege Verläufe nur an Orten ab, die vor unbefugtem Zugriff geschützt sind, und lösche sie, wenn sie nicht mehr gebraucht werden.

Typische Fehler

Der häufigste Fehler ist, den gesamten Verlauf ohne Begrenzung mitzusenden. Das Kontextfenster läuft dann voll, die Antworten werden langsam oder abgeschnitten. Ein zweiter ist, den Systemprompt zu verlängern, bis er widersprüchlich ist. Ein dritter ist, Verläufe unverschlüsselt und dauerhaft auf einem geteilten Rechner abzulegen.

Kurz gesagt

Ein Assistent besteht aus Modell, Systemprompt, begrenztem Verlauf und sicherer Ablage. Das Gedächtnis ist eine Frage der Programmierung, nicht des Modells.

Übungen

Übung 5.1: Ändere die Begrenzung auf zwei Nachrichten und beschreibe, was der Assistent bei der dritten Frage noch sieht.

Übung 5.2: Formuliere einen Systemprompt für einen Assistenten, der Protokolle von Besprechungen zusammenfasst. Nenne drei Regeln, die für alle Gespräche gelten sollen.

Kapitel 6: Dokumente mit lokaler KI verarbeiten

6.1 Lokale Dokumentenanalyse

Dokumente lokal zu verarbeiten ist der häufigste Grund, ein eigenes System aufzubauen. Verträge, interne Richtlinien und Berichte dürfen oft nicht an externe Dienste gehen. Die Analyse folgt denselben Regeln wie in Band 2: Aussagen brauchen Fundstellen, und fehlende Informationen müssen als fehlend gekennzeichnet werden.

6.2 RAG-Systeme

Band 2 hat das Prinzip von RAG erklärt: Relevante Abschnitte werden abgerufen und dem Modell als Kontext übergeben. Ein lokales RAG-System besteht aus denselben Schritten, nur läuft alles auf deinem Rechner. Die Schritte sind Aufnehmen der Dokumente, Zerlegen in Abschnitte, Berechnen von Vektoren, Speichern, Suchen und Antworten mit Quellenangabe.

6.3 Embedding-Modelle

Für echte Suche nach Bedeutung braucht das System ein Embedding-Modell, das Text in Vektoren verwandelt. Solche Modelle sind kleiner als Sprachmodelle und laufen auch auf schwacher Hardware. Viele Modellserver bieten Embeddings über eine eigene Schnittstelle an. Der Endpunkt und die Form der Antwort ändern sich zwischen Versionen und stehen in der Dokumentation.

Das folgende Beispiel verwendet kein Embedding-Modell, sondern zählt identische Wörter. Dieser Platzhalter lässt sich ohne Modell testen, und er zeigt genau, wo ein echtes Embedding ansetzt. Er findet nur Stellen, in denen dieselben Wörter vorkommen. Die Ergebnisse im

Test machen das sichtbar.

6.4 Vektordatenbanken

Vektoren lassen sich in einer Datenbank speichern, die auf Ähnlichkeitssuche spezialisiert ist. Für kleine Sammlungen mit einigen tausend Abschnitten genügt eine einfache Datenbank wie SQLite, die zum Lieferumfang von Python gehört. Wächst die Sammlung, lohnt sich eine spezialisierte Vektordatenbank. Sie bietet schnellere Suche und Funktionen für Filter und Aktualisierung.

6.5 Lokale Wissensdatenbanken

Eine lokale Wissensdatenbank besteht aus den Dokumenten, dem Index mit Vektoren und den Metadaten, etwa Titel, Datum und Quelle. Der Index muss aktualisiert werden, wenn sich Dokumente ändern. Lösche Einträge zu entfernten Dokumenten, sonst liefert das System veraltete Antworten. Eine einfache Regel lautet, bei jeder Aufnahme die Datei nach Änderungsdatum und Prüfsumme zu vergleichen.

6.6 Das RAG-Beispiel

Das folgende Modul nimmt vier kurze Dokumente auf, zerlegt sie, speichert die Vektoren in einer SQLite-Datenbank, sucht passende Abschnitte und baut einen Prompt, der nur mit diesen Abschnitten antworten soll. Das Modell im Test ist ein Platzhalter. Die Abschnitte und der Prompt sind die eigentliche Arbeit, und genau diese lassen sich prüfen.

Codebeispiel 6.1: Lokale RAG-Pipeline mit SQLite

```
import math
import re
import sqlite3
import zlib

from lokal_client import chat

DIMENSION = 512


def chunks(text, groesse=60, ueberlappung=15):
    woerter = text.split()
    schritt = groesse - ueberlappung
    ende = max(len(woerter) - ueberlappung, 1)
    return [" ".join(woerter[i:i + groesse])
            for i in range(0, ende, schritt)]


def embed(text):
    # Platzhalter fuer ein echtes Embedding-Modell: zaehlt
    # identische Woerter in einen festen Vektor.
    vektor = [0.0] * DIMENSION
    for wort in re.findall(r"\w+", text.lower()):
        stelle = zlib.crc32(wort.encode("utf-8")) % DIMENSION
        vektor[stelle] += 1.0
    norm = math.sqrt(sum(x * x for x in vektor)) or 1.0
    return [x / norm for x in vektor]


def cosinus(a, b):
    return sum(x * y for x, y in zip(a, b))


class Speicher:
    def __init__(self, pfad=":memory:"):
        self.db = sqlite3.connect(pfad)
        self.db.execute(
            "CREATE TABLE IF NOT EXISTS chunks "
            "(id INTEGER PRIMARY KEY, quelle TEXT, "
            "text TEXT, vektor TEXT)")

    def aufnehmen(self, quelle, text):
        for stueck in chunks(text):
            self.db.execute(
                "INSERT INTO chunks (quelle, text, vektor) "
                "VALUES (?, ?, ?)",
                (quelle, stueck, str(embed(stueck))))
        self.db.commit()
```

```
    def suchen(self, frage, k=2):
        q = embed(frage)
        treffer = []
        zeilen = self.db.execute(
            "SELECT quelle, text, vektor FROM chunks")
        for quelle, text, vektor in zeilen:
            zahlen = vektor.strip("[]").split(",")
            v = [float(x) for x in zahlen]
            treffer.append((cosinus(q, v), quelle, text))
        treffer.sort(reverse=True)
        return treffer[:k]


def prompt_bauen(frage, treffer):
    quellen = "\n".join(
        f"[Quelle {i}] ({t[1]}) {t[2]}"
        for i, t in enumerate(treffer, 1))
    return ("Beantworte die Frage nur mit den Quellen unten. "
            "Nenne die Quellennummer. Wenn die Quellen die Antwort "
            "nicht enthalten, schreibe: nicht in den Quellen.\n\n"
            f"{quellen}\n\nFrage: {frage}")


def fragen(modell, speicher, frage):
    treffer = speicher.suchen(frage)
    prompt = prompt_bauen(frage, treffer)
    antwort = chat(modell, [{"role": "user", "content": prompt}])
    return antwort, treffer
```

Ausgabe

```
Frage: Wie lange ist die Kündigungsfrist?
  0.42 kuendigung.txt
  0.10 urlaub.txt
   Platzhalter: 1 Nachrichten, System nein, letzte 541 Zeichen
Frage: Wie viele Urlaubstage habe ich?
  0.09 wlan.txt
  0.00 urlaub.txt
   Platzhalter: 1 Nachrichten, System nein, letzte 516 Zeichen
Frage: Wie oft erhalte ich Geld für Reisen?
  0.08 urlaub.txt
  0.08 wlan.txt
   Platzhalter: 1 Nachrichten, System nein, letzte 521 Zeichen
```

Die Ausgabe zeigt für jede Frage die besten Abschnitte mit ihrem Ähnlichkeitswert. Die erste Frage findet das richtige Dokument, weil das Wort Kündigungsfrist in Frage und Dokument identisch ist. Die zweite Frage findet das Urlaubsdokument nicht an erster Stelle, weil in

der Frage das Wort Urlaubstage steht, im Dokument aber Urlaub und Urlaubsanspruch. Die dritte Frage verwendet Geld und Reisen, das Dokument spricht von Reisekosten, die erstattet werden. Ein Wortzähler erkennt diese Zusammenhänge nicht. Ein echtes Embedding-Modell tut es in der Regel, weil es Bedeutung über Wortgrenzen hinweg abbildet.

6.7 Grenzen einfacher Verfahren

Der Test zeigt eine wichtige Lektion. Eine Pipeline kann technisch vollständig sein und trotzdem schlechte Ergebnisse liefern, wenn die Suche die falschen Abschnitte findet. Das Prompt-Gerüst schützt dann nicht, denn das Modell antwortet mit dem, was ihm vorliegt. Deshalb gehört zu jedem RAG-System eine Prüfung der Treffer, bevor man einer Antwort vertraut. Die Quellenangabe im Prompt hilft dabei, sie nachzuschlagen.

Achtung: Vertrauliche Dokumente im Index

Der Index enthält Auszüge der Dokumente im Klartext. Schütze die Datenbankdatei wie die Dokumente selbst, mit Zugriffsrechten und Verschlüsselung der Festplatte, und lege sie nicht in öffentlich erreichbare Ordner.

Typische Fehler

Der häufigste Fehler ist, die Qualität der Treffer nicht zu prüfen und nur die Antworten zu lesen. Ein zweiter ist, Dokumente ohne Metadaten aufzunehmen, sodass Quellen später nicht mehr nachvollziehbar sind. Ein dritter ist, den Index nie zu aktualisieren.

Kurz gesagt

Lokales RAG besteht aus Aufnahme, Zerlegung, Vektoren, Speicherung, Suche und Antwort. Die Qualität der Treffer entscheidet über die Qualität der Antwort, und ein echtes Embedding-Modell verbessert die Suche deutlich.

Übungen

Übung 6.1: Ersetze im Beispiel die Frage nach Urlaubstage durch Urlaub beantragen und vergleiche die Treffer. Woran liegt der Unterschied?

Übung 6.2: Nimm ein weiteres Dokument auf, das die Reisekostenregeln mit anderen Worten erklärt, und prüfe, ob die dritte Frage danach besser beantwortet wird.

Kapitel 7: Betrieb und Administration

7.1 Linux-Server

Ein Server, der ein Modell dauerhaft bereitstellt, braucht eine feste Umgebung. Viele Administratoren wählen dafür Linux, weil es stabil läuft, wenig Ressourcen braucht und sich gut automatisieren lässt. Die Distribution Debian ist dafür verbreitet. Die Einrichtung umfasst Systemaktualisierungen, eine Firewall, einen Benutzer für den Dienst und einen Dienst, der beim Systemstart automatisch läuft.

Dienste verwaltet unter Debian und vielen anderen Distributionen das Programm systemd. Ein Dienst wird über eine Konfigurationsdatei beschrieben, die festlegt, wie er startet, welcher Benutzer ihn ausführt und wie er bei Fehlern neu startet. Die genauen Einstellungen für einen Modellserver stehen in dessen Dokumentation. Dieses Buch nennt keine Vorlage, weil sie nicht überprüft werden konnte.

7.2 Docker

Docker verpackt Programme zusammen mit ihrer Umgebung in Containern. Ein Container läuft isoliert vom Rest des Systems, lässt sich leicht aktualisieren und ist mit wenigen Befehlen wieder entfernt. Für Oberflächen wie Open WebUI ist das praktisch, weil die Installation dann nicht die Systemumgebung verändert.

Daten in einem Container gehen beim Entfernen verloren, wenn du sie nicht in einen Datenträger außerhalb legst. Solche Datenträger heißen Volumes oder Einhängepunkte. Sie müssen beim Anlegen des Containers ausdrücklich festgelegt werden, und genau dort liegt eine häufige Fehlerquelle.

7.3 Netzwerkzugriff

Ein lokales System sollte zunächst nur im eigenen Netz erreichbar sein. Die Firewall des Servers erlaubt nur die Ports, die tatsächlich gebraucht werden. Ein Zugriff von außen über das Internet ist nur mit einem sicheren Zugang möglich, etwa über ein VPN, das den Zugang auf berechtigte Geräte beschränkt. Ein ungeschützter Modellserver im Internet ist ein erhebliches Risiko.

Ein Reverse Proxy kann einen Dienst mit Verschlüsselung und Anmeldung vor dem direkten Zugriff schützen. Die Einrichtung erfordert Erfahrung mit Zertifikaten und Zugriffsregeln. Wenn du sie nicht selbst leisten kannst, ist ein VPN oft der einfachere Weg.

Achtung: Portfreigaben im Router

Eine Weiterleitung eines Ports aus dem Internet auf den Server macht den Dienst für jeden erreichbar, der die Adresse findet. Richte eine solche Freigabe nur ein, wenn du den Zugriff zuverlässig absicherst, und prüfe die Einstellungen regelmäßig.

7.4 Benutzerverwaltung

Jeder Mensch, der das System nutzt, bekommt ein eigenes Konto mit den Rechten, die er braucht. Administratoren haben erhöhte Rechte, alle anderen nicht. Gemeinsam genutzte Zugangsdaten verhindern nachvollziehbare Zuordnung und sollten vermieden werden. Entferne Konten, sobald Personen das System nicht mehr brauchen.

7.5 Backups

Ein Backup ist nur dann ein Backup, wenn du es wiederherstellen kannst. Sichere deshalb nicht nur Konfigurationen, sondern auch die Datenbanken, die Verläufe und den Index. Die Modelldateien sind groß und lassen sich bei Bedarf neu herunterladen, sie gehören aber in die Planung des Speicherplatzes. Eine verbreitete Regel lautet, drei Kopien auf zwei verschiedenen Medien zu halten, davon eine an einem anderen Ort.

Teste die Wiederherstellung einmal im Jahr auf einem zweiten Gerät. Ein Backup, das nie geprüft wurde, ist eine Hoffnung und keine Sicherung.

7.6 Monitoring

Das Monitoring zeigt, ob das System gesund ist. Beobachte den freien Speicherplatz auf der Festplatte, die Auslastung von Prozessor und Speicher, die Temperatur und die Fehlermeldungen im Protokoll. Ein Dienst, der regelmäßig neu startet, zeigt ein Problem an, das sich mit einem Blick in das Protokoll meist findet.

7.7 Datenschutz und Dokumentation

Die Datenschutz-Grundverordnung verlangt, dass personenbezogene Daten durch geeignete technische und organisatorische Maßnahmen geschützt werden, Artikel 32. Ein lokales System ist davon nicht ausgenommen. Dokumentiere, welche Daten verarbeitet werden, wer Zugriff hat, wie lange Verläufe gespeichert bleiben und wie gesichert wird. Diese Dokumentation hilft dir auch, wenn du die Einrichtung später erklären oder übergeben musst.

Schritt für Schritt: Einen lokalen Server betriebsbereit machen

1. Aktualisiere das System und lege einen Benutzer für den Modelldienst an.
2. Richte die Firewall so ein, dass nur die nötigen Ports erreichbar sind.
3. Konfiguriere den Dienst so, dass er beim Start automatisch läuft und bei Fehlern neu startet.
4. Lege Datenordner außerhalb der Container an und binde sie ein.
5. Richte ein Backup der Daten ein und teste die Wiederherstellung auf einem zweiten Gerät.
6. Dokumentiere Zweck, Daten, Zugriffe und Aufbewahrungsfristen.

Erwartetes Ergebnis: Ein Server, der nach einem Neustart selbstständig läuft, nur die nötigen Zugänge freigibt und dessen Wiederherstellung geprüft ist.

Typische Fehler

Der häufigste Fehler im Betrieb ist, Backups einzurichten und nie zu prüfen. Ein zweiter ist, Container ohne Datenordner zu betreiben und beim Aktualisieren alle Verläufe zu verlieren. Ein dritter ist, Ports zu öffnen, um eine Verbindung schnell zum Laufen zu bringen, und diese Freigabe nie wieder zu prüfen.

Kurz gesagt

Betrieb heißt Sicherheit, Verfügbarkeit und Nachvollziehbarkeit. Ein Server gehört in ein abgesichertes Netz, seine Daten gehören in ein geprüftes Backup, und alles gehört in eine Dokumentation.

Übungen

Übung 7.1: Erstelle eine Liste aller Ports, die dein Server offen hat. Begründe jeden Port oder schließe ihn.

Übung 7.2: Plane die Backups für deinen Server: Was wird gesichert, wie oft, wohin, und wann prüfst du die Wiederherstellung?

Kapitel 8: Praxisprojekte

8.1 Hinweise zu den Projekten

Die drei Projekte bauen auf den Kapiteln 1 bis 7 auf. Das erste und das dritte brauchen nur einen Rechner mit Python und einem Modellserver. Das zweite erfordert zusätzlich Dokumente, mit denen du arbeiten willst. Verwende für alle drei zunächst Beispieldaten, bevor du echte Unterlagen einbindest. Ein Projekt gilt als abgeschlossen, wenn das erwartete Ergebnis vorliegt und ein Fehlerfall absichtlich ausgelöst und geprüft wurde.

8.2 Projekt 1: Eigener lokaler Chatbot

Voraussetzungen und Werkzeuge

1. Ein Rechner mit Python 3 und einem lokalen Modellserver mit einem installierten Modell.
2. Die Module lokal_client.py und gedaechtnis.py aus dem Codebeispiel.
3. Ein Textverzeichnis für gespeicherte Verläufe.

Hinweis: Der Chatbot läuft im Terminal und speichert seinen Verlauf auf Wunsch.

Durchführung in fünf Schritten

1. Prüfe mit der Modellliste, dass das gewünschte Modell verfügbar ist.

2. Schreibe ein kleines Programm, das in einer Schleife Eingaben liest und an das Gespräch übergibt.

3. Lege einen Systemprompt für deinen Anwendungsfall fest und begrenze den Verlauf auf einen sinnvollen Wert.

4. Speichere den Verlauf beim Beenden und lade ihn beim nächsten Start auf Wunsch.

5. Prüfe, dass ein Fehler beim Modell verständlich gemeldet wird und das Programm nicht abstürzt.

Erwartetes Ergebnis: Ein Chatbot im Terminal mit eigenem Systemprompt, begrenztem Verlauf, Speicherung und sauberer Fehlermeldung.

Fehlerbehebung: Antwortet der Chatbot nicht, prüfe zuerst die Adresse des Servers und die Modellliste. Vergisst er frühere Fragen, ist die Begrenzung zu klein gewählt. Sicherheitshinweis: Verläufe enthalten Eingaben von dir. Speichere sie nur an einem geschützten Ort.

8.3 Projekt 2: Lokaler Dokumentenassistent

Voraussetzungen und Werkzeuge

1. Ein Rechner mit Python 3 und dem Modul rag_lokal.py aus dem Codebeispiel.

2. Ein Ordner mit mindestens zehn Textdokumenten zu einem Thema, das du kennst.

3. Ein Modellserver mit einem Sprachmodell für die Antworten.

Hinweis: Ein lokaler Assistent, der Fragen zu deinen Dokumenten mit Quellenangabe beantwortet.

Durchführung in fünf Schritten

1. Lies die Dokumente aus dem Ordner ein und nimm sie mit Dateiname als Quelle in den Speicher auf.

2. Stelle zehn Fragen, deren Antworten du kennst, und prüfe, ob die Treffer die richtige Quelle enthalten.

3. Notiere alle Fragen, bei denen der falsche Abschnitt an erster Stelle steht, und suche nach dem Grund.

4. Verbessere die Zerlegung, etwa durch andere Abschnittsgrößen, und wiederhole den Test.

5. Ersetze den Wortzähler durch ein Embedding-Modell, sobald dein Modellserver es anbietet, und vergleiche die Ergebnisse.

Erwartetes Ergebnis: Ein Assistent mit dokumentierter Trefferquote, der die Grenzen des einfachen Verfahrens zeigt und durch ein Embedding-Modell verbessert werden kann.

Fehlerbehebung: Wenn die Treffer schlecht sind, prüfe die Zerlegung und die Wortwahl der Fragen. Wenn das Modell Informationen erfindet, obwohl die Quellen sie nicht enthalten, verschärfe die Anweisung im Prompt und prüfe die Antwort gegen die Quelle. Sicherheitshinweis: Der Index enthält Textauszüge. Schütze ihn wie die Dokumente selbst.

8.4 Projekt 3: Persönlicher KI-Assistent mit Gedächtnis

Im dritten Projekt verbindest du Chatbot und Dokumentensuche zu einem persönlichen Assistenten. Er merkt sich Gespräche über mehrere Sitzungen, durchsucht deine eigenen Notizen und antwortet mit Quellenangabe. Bei diesem Projekt ist der Schutz der Daten am wichtigsten, deshalb beginnt die Arbeit mit einer Festlegung, welche Daten hineindürfen.

Durchführung in fünf Schritten

1. Lege schriftlich fest, welche Daten in den Assistenten dürfen und welche nicht, und wie lange Verläufe bleiben.
2. Kombiniere den Verlauf aus Projekt 1 mit der Suche aus Projekt 2, sodass jede Antwort beide Quellen nutzen kann.
3. Speichere Verläufe und Index in getrennten Dateien mit eingeschränkten Zugriffsrechten.
4. Teste eine Sitzung, beende sie, starte neu und prüfe, ob der Verlauf und die Suche erhalten bleiben.
5. Lösche in einem Test alle gespeicherten Daten und prüfe, dass der Assistent danach nichts mehr kennt.

Erwartetes Ergebnis: Ein persönlicher Assistent, dessen Gedächtnis, Suche und Löschung nachvollziehbar und geprüft sind.

Fehlerbehebung: Verliert der Assistent frühere Informationen, ist das Speichern nicht vollständig. Zeigt er Inhalte aus alten Sitzungen, die nicht mehr relevant sind, begrenze den Verlauf. Sicherheitshinweis: Ein Assistent mit Zugriff auf Notizen ist ein sensibles System. Betreibe ihn nur auf einem Gerät, das du selbst kontrollierst.

Kurz gesagt

Die drei Projekte führen von einem Chatbot über eine Dokumentensuche zu einem persönlichen Assistenten. Jedes Projekt zeigt, dass der Aufwand weniger im Modell liegt als in Verlauf, Suche, Schutz und Prüfung.

Übungen

Übung 8.1: Führe Projekt 1 durch und notiere, nach wie vielen Nachrichten der Chatbot ältere Angaben vergisst.

Übung 8.2: Miss in Projekt 2 für zehn Fragen, wie oft der richtige Abschnitt an erster Stelle steht. Notiere die Trefferquote.

Musterlösungen

Die folgenden Lösungen sind Vorschläge. Bei den Übungen gibt es keine einzig richtige Antwort, entscheidend ist die Begründung und die Prüfung am eigenen Material.

Kapitel 1

Lösung 1.1: Eine vollständige Notiz nennt die Datenart, ihre Schutzbedürftigkeit, etwa Personenbezug oder Geschäftsgeheimnis, und die Folgen einer Veröffentlichung.

Lösung 1.2: Wichtige Bedingungen sind meist die Zulässigkeit kommerzieller Nutzung, Weitergabe und Pflichten bei Veröffentlichung von Ergebnissen. Die Angaben stehen in der jeweiligen Lizenz.

Kapitel 2

Lösung 2.1: Mit der Faustregel ergibt sich 13 mal 8 geteilt durch 8, also 13 Gigabyte, mal 1,2, also etwa 15,6 Gigabyte. Der Wert ist ein Überschlag und keine Messung.

Lösung 2.2: Ein Rechner ist geeignet, wenn sein Speicher für die Gewichtungen mit Puffer ausreicht. Bei knappem Speicher sind eine kleinere Quantisierung oder ein anderes Modell zu prüfen.

Kapitel 3

Lösung 3.1: Eine vollständige Liste enthält die Installationsquelle, die verwendete Version und die Prüfung des Serverdienstes.

Lösung 3.2: Die Messung zeigt im Leerlauf meist einen niedrigen Verbrauch und während einer Anfrage einen deutlich höheren. Die Differenz ist die Grundlage für die Planung.

Kapitel 4

Lösung 4.1: Mit fünf Sekunden bricht der Client bei einem nicht erreichbaren Server früher ab. Die Fehlermeldung bleibt dieselbe, die Wartezeit wird kürzer.

Lösung 4.2: Eine Tabelle ordnet jedem Modell eine Aufgabe und einen Speicherbedarf zu. Das größere Modell übernimmt anspruchsvolle Aufgaben, das kleinere einfache Fragen mit kurzer Antwortzeit.

Kapitel 5

Lösung 5.1: Mit zwei Nachrichten sieht der Assistent bei der dritten Frage nur den Systemprompt und die letzte Frage, nicht mehr die erste Antwort.

Lösung 5.2: Ein Systemprompt für Protokolle nennt Ziel, Länge und Form der Zusammenfassung sowie den Hinweis, nichts zu erfinden, was im Protokoll fehlt.

Kapitel 6

Lösung 6.1: Das Wort Urlaub kommt im Dokument vor, Urlaubstage dagegen nicht. Der Wortzähler sieht nur identische Wörter und findet deshalb das Urlaubsdokument mit der genauen Formulierung.

Lösung 6.2: Ein weiteres Dokument mit Reisekostenbegriffen kann die Trefferquote verbessern, weil es die Wörter der Frage enthält. Ein echtes Embedding-Modell wäre dafür eine bessere Lösung.

Kapitel 7

Lösung 7.1: Für jeden offenen Port muss ein Zweck bestehen. Ports ohne Zweck sind zu schließen, und der Zugang für den Modellserver sollte auf das Netz beschränkt sein.

Lösung 7.2: Ein Backupplan nennt Datenarten, Häufigkeit, Speicherort außerhalb des Servers und einen festen Termin für die Wiederherstellungsprüfung.

Kapitel 8

Lösung 8.1: Der Chatbot vergisst ältere Angaben bei der Grenze, die im Verlauf festgelegt ist. Die Zahl ergibt sich aus der Einstellung.

Lösung 8.2: Eine Trefferquote von zehn Fragen ist klein, aber für den Vergleich verschiedener Zerlegungen aussagekräftig. Notiere die Fragen mit Fehltreffern.

Anhang: Betriebscheckliste für lokale KI-Systeme

Gehe diese Checkliste durch, bevor ein lokales System Mitarbeitende oder Dritte erreicht.

- Daten: Ist festgelegt, welche Daten das System verarbeiten darf?
- Lizenz: Ist die Lizenz jedes eingesetzten Modells geprüft?
- Hardware: Ist der Speicherbedarf gemessen und mit Puffer geplant?
- Netz: Ist der Modellserver nur in dem Netz erreichbar, das vorgesehen ist?
- Firewall: Sind nur die nötigen Ports offen?
- Benutzer: Hat jede Person ein eigenes Konto mit den nötigen Rechten?
- Dienst: Startet der Dienst nach einem Neustart automatisch?
- Daten im Container: Liegen Verläufe und Index in eingebundenen Ordnern?
- Backup: Wurde die Wiederherstellung auf einem zweiten Gerät geprüft?
- Monitoring: Werden Speicherplatz, Auslastung und Fehlerprotokoll beobachtet?
- Dokumentation: Sind Zweck, Daten, Zugriffe und Aufbewahrungsfristen festgehalten?

Glossar

Backup: Gesicherte Kopie von Daten und Konfigurationen, die sich wiederherstellen lassen muss.

Container: Isoliert laufende Umgebung für ein Programm, die mit allen Abhängigkeiten verpackt ist, etwa über Docker.

Embedding: Umwandlung eines Textes in einen Vektor aus Zahlen, dessen Nähe die Ähnlichkeit der Bedeutung abbildet.

Gewichtungen: Zahlenwerte, die das Verhalten eines Sprachmodells bestimmen und in der Modelldatei gespeichert sind.

GPU: Grafikprozessor, der parallele Rechnungen schnell ausführt und sich für Sprachmodelle eignet.

Kontextfenster: Menge an Text, die ein Modell bei einer Anfrage gleichzeitig berücksichtigen kann.

Modellkarte: Beschreibung eines Modells mit Angaben zu Trainingsdaten, Einschränkungen und vorgesehener Nutzung.

Open-Weight: Bezeichnung für Modelle, deren Gewichtungen öffentlich verfügbar sind, unabhängig von der Lizenz im Einzelnen.

Parameter: Die einstellbaren Zahlenwerte eines Modells, meist in Milliarden angegeben.

Quantisierung: Speicherung der Gewichtungen mit weniger Bits, um den Speicherbedarf zu senken.

RAG: Retrieval-Augmented Generation. Abruf passender Abschnitte, die dem Modell als Kontext mitgegeben werden.

Reverse Proxy: Vorgeschalteter Server, der Anfragen entgegennimmt, prüft und an den eigentlichen Dienst weiterleitet.

Systemprompt: Anweisung, die bei jeder Anfrage mitgesendet wird und Rolle und Grundregeln eines Assistenten festlegt.

systemd: Dienstverwaltung vieler Linux-Distributionen, die Dienste beim Start aktiviert und überwacht.

Temperatur: Parameter, der die Zufälligkeit der Antworten steuert. Niedrige Werte sind gleichmäßiger.

VRAM: Speicher einer Grafikkarte, in dem das Modell für schnelle Berechnungen liegen sollte.

Volume: Datenordner außerhalb eines Containers, der Daten bei einem Neustart oder Entfernen des Containers erhält.

Vektordatenbank: Datenbank, die Vektoren speichert und die ähnlichsten Einträge zu einem Vektor findet.

Quellenverzeichnis

Die folgenden Quellen bilden die Grundlage für die rechtlichen und methodischen Angaben dieses Bandes. Bei Gesetzen gilt stets die aktuell geltende Fassung.

Verordnung (EU) 2016/679 des Europäischen Parlaments und des Rates (Datenschutz-Grundverordnung), Amtsblatt der Europäischen Union L 119 vom 4. Mai 2016, insbesondere Artikel 5 und Artikel 32.

Verordnung (EU) 2024/1689 des Europäischen Parlaments und des Rates über künstliche Intelligenz (KI-Verordnung), Amtsblatt der Europäischen Union, 2024.

Dokumentation des Modellservers Ollama, insbesondere zur Schnittstelle, zu Befehlen und zu Modellparametern. Projektseiten auf GitHub, Stand bei jeder Nutzung prüfen.

Projektdokumentation von llama.cpp und dem GGUF-Format zur Quantisierung und Speicherung von Modellen. Stand bei jeder Nutzung prüfen.

Dokumentation von Docker zu Containern und Volumes sowie von systemd zu Diensten unter Linux. Stand bei jeder Nutzung prüfen.

Lewis, Patrick u. a.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: Advances in Neural Information Processing Systems 33, 2020.

Dokumentation der SQLite-Datenbank zum Modul sqlite3 der Python-Standardbibliothek.

Stichwortverzeichnis

Eine Auswahl wichtiger Begriffe mit Verweisen auf die Kapitel. Die Angaben beziehen sich auf die Kapitelnummern dieses Bandes.

Begriff	Kapitel
Backup	7
Container	7
Embedding	6
Gewichtungen	1, 2
GPU	2
Kontextfenster	2, 5
Lizenz	1
Open-Weight	1
Parameter	2, 3
Quantisierung	2
RAG	6
Reverse Proxy	7
Systemprompt	5
Temperatur	3
VRAM	2
Vektordatenbank	6

KI PRAXIS

Band 4 zeigt, wie du Sprachmodelle auf eigener Hardware betreibst und daraus Assistenten baust, die mit deinen Dokumenten arbeiten. Du lernst, Hardware einzuschätzen, einen Modellserver zu nutzen, ein Gespräch mit Gedächtnis zu führen und eine RAG-Pipeline zu verstehen.

Die Codebeispiele sind ausgeführt und mit ihren Ausgaben abgedruckt. Das Buch benennt ehrlich, wo einfache Verfahren an Grenzen stoßen, und zeigt, wie der Betrieb abgesichert wird.

Die weiteren Bände der Reihe KI PRAXIS: Band 1 KI im produktiven Alltag, Band 2 Recherche und Wissensmanagement, Band 3 Automatisierung und KI-Agenten, Band 5 Datenanalyse mit KI, Band 6 Dokumente, Präsentationen und Fachbücher, Band 7 Lernen mit KI und Band 8 Sicherheit und verantwortungsvolle Nutzung.

Eugen Becker | 1. Auflage, November 2026